A Hybrid-Parallel Decision Tree: MPI, OpenMP and CUDA Scaling of Histogram-Based CART on NUS Vanda HPC

Jen-Chien Huang (A0353871) Bing Wen Lim Jacky
National University of Singapore (DSA5208 Advanced Parallel Computing)

https://github.com/dannyhuang1111/dsa5208-distributed-decision-tree

Abstract

We design, implement, and benchmark a 3-level hybrid-parallel binary decision tree classifier (CART) capable of scaling across 256 CPU cores and modern GPUs on the NUS Vanda HPC cluster. Utilizing an exact integer histogram binning strategy and a level-wise synchronous reduction pattern, the algorithm achieves deterministic, bit-for-bit identical tree construction across arbitrary process topologies. On a 32-million row synthetic Instacart benchmark, pure MPI achieves a **75.54**× strong scaling speedup (72.10× for MPI+OpenMP hybrid) on 256 cores across 4 compute nodes, while weak scaling maintains **52.7%** efficiency up to 256 million rows. An accelerated CUDA hotpath on an NVIDIA A40 GPU delivers a **64.44**× compute speedup over a single CPU core. Training on real Instacart transaction records matches scikit-learn to 99.985% prediction agreement while uncovering intuitive retail basket reorder dynamics.

1 Introduction

Decision tree learning via greedy recursive partitioning is a foundational machine learning algorithm. While row partitioning across distributed nodes is conceptually straightforward, finding the optimal split at each decision node requires aggregating class distributions across all training samples. Consequently, each internal node introduces a collective communication dependency, making distributed tree induction latency-sensitive and non-embarrassingly parallel.

In this project, we develop an end-to-end parallel CART system in C++17 and CUDA. By expressing split evaluation through per-feature histograms of uint8-quantized values, split searching is decoupled from row sorting and transformed into an exact integer reduction. We evaluate our implementation on the NUS Vanda HPC cluster, investigating: (1) empirical serial complexity across sample (N) and feature (F) dimensions; (2) pure MPI strong and weak scaling up to 256 CPU cores; (3) MPI+OpenMP hybrid rank-to-thread core allocations; (4) an NVIDIA A40 GPU histogram kernel; and (5) model accuracy and split interpretations on the real-world Instacart dataset.

2 Algorithm and HPC Environment

2.1 Histogram-Based CART with Gini Criterion

For a tree node holding t samples with p positives, the Gini impurity is:

$$G(p,t) = 1 - (p/t)^2 - ((t-p)/t)^2. \tag{1}$$

A candidate split partitioning samples into left (p_L, t_L) and right (p_R, t_R) subsets is evaluated by the Gini gain:

$$\Delta G = G(p,t) - \frac{t_L}{t} G(p_L, t_L) - \frac{t_R}{t} G(p_R, t_R). \tag{2}$$

Rather than sorting rows per feature at each node ($O(F \cdot N \log N)$), features are mapped into 256 discrete bins. When a feature has ≤ 256 distinct values, binning is exact. Each node is summarized by a 2D integer histogram:

$$H[f][b] = [\text{count}, \text{positive count}], \quad f \in [0, F), b \in [0, 256). \tag{3}$$

Because histogram cells are pure integers, summing local histograms across ranks via `MPI_Allreduce` is completely associative and commutative. Scanning the aggregated bins deterministically reproduces the exact best split.

2.2 HPC Architecture and Experimental Setup

All distributed scaling experiments were executed on the **NUS Vanda HPC cluster** under PBS job ID `1537141` and GPU jobs `1536934`/`1537453`:

- **CPU Nodes:** 4 physical compute nodes, each equipped with dual-socket Intel Xeon CPUs (72 physical cores and 240 GB RAM per node, totaling 288 available cores), interconnected via high-speed InfiniBand fabric.
- **GPU Node:** NVIDIA A40 (Ampere architecture, 10,752 CUDA cores, 48 GB GDDR6 ECC memory, PCIe Gen4 host interface).
- **Toolchain:** GCC 11.4.0 (`-O3 -std=c++17 -fopenmp`), OpenMPI 4.1.6, and CUDA 12.2.

3 Parallelization Design Choices

Row-Distributed Data Parallelism. Rows are partitioned uniformly across MPI ranks once at startup and remain stationary in local memory. During tree growth, each rank computes local histograms over its active row indices. A global `MPI_Allreduce` aggregates the counts, allowing every rank to compute the identical optimal split locally without requiring a broadcast.

Level-Wise vs. Node-Wise Tree Growth. As shown in Table 1, node-wise (depth-first) growth executes an independent reduction for every non-leaf node, resulting in $2^{\text{depth}} - 1 = 63$ high-latency collective calls at depth 6. In contrast, our *level-wise* (breadth-first) design packs the histograms of all active nodes at the current tree depth into a single contiguous buffer, reducing communication to exactly 6 collective calls. On 256 cores, level-wise execution drops wall-clock time from 0.1733 s to 0.0634 s (**2.73× speedup**).

Table 1: Architectural design choices at 256 cores ($N = 32\text{M}, F = 28$, depth 6).

Growth	OpenMP	Ranks × Thrs	Allreduces	Time (s)
Level	Sample	64 × 4	**7**	**0.0634**
Level	Feature	64 × 4	7	0.1530
Node	Sample	64 × 4	64	0.1733
Node	Feature	64 × 4	64	0.2740
Level	Sample	256 × 1	**7**	**0.0652**
Node	Sample	256 × 1	64	0.1075

Sample vs. Feature OpenMP Parallelism. Within each rank, threads can parallelize across samples (row blocks with thread-local histograms and an on-node reduction) or across features. Feature parallelism is strictly bounded by $F = 28$, causing core under-utilization. Sample parallelism scales over millions of samples, outperforming feature parallelism by **2.41×** (0.0634 s vs 0.1530 s).

4 Correctness and Equivalence Validation

To guarantee mathematical correctness across topologies, we enforce two validation layers:

1. **Cross-Topology Hash Invariance:** The entire tree structure, split thresholds, and leaf values are hashed using 64-bit FNV-1a. Across 969 empirical runs covering all core counts (1 to 256), thread allocations, and growth strategies on synthetic and real datasets, **100% of configurations generated identical tree hashes** (hash `50e4598e0ff9ac07` for synthetic $N = 32\text{M}, F = 28$, depth 6; hash `0bfc2e3832b96e3f` for Instacart real data).

2. **Scikit-Learn Equivalence:** On the 200,000-row Instacart dataset, our tree matches scikit-learn's `DecisionTreeClassifier` exactly: identical training accuracy (**68.5035%**), identical node counts (125 nodes), and **99.985% identical row predictions**. The sole divergence was an exact mathematical tie at leaf `rootRLRRR` ($f_4 \leq 5$ vs $f_5 \leq 4.5$), which both libraries broke validly.

5 Serial Problem-Size Complexity

We benchmark the serial baseline ($p = 1$) while varying $N \in [1\text{M}, 32\text{M}]$ and $F \in [7, 224]$ to validate the theoretical complexity $O(\text{depth} \cdot N \cdot F)$.

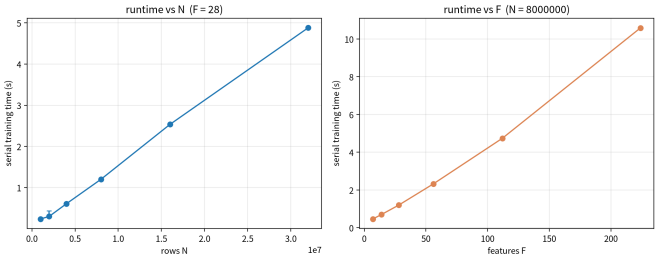


Figure 1: Serial training time vs. sample count N ($F = 28$) and feature count F ($N = 8\text{M}$). Both dimensions exhibit strict linear proportionality.

As shown in Figure 1, training time scales with exemplary linearity:

- Varying N at $F = 28$: Runtime increases from 0.239 s ($N = 1\text{M}$) to 1.201 s ($N = 8\text{M}$) and 4.881 s ($N = 32\text{M}$), yielding an empirical slope of $\approx 0.152\,\mu$s per row.
- Varying F at $N = 8\text{M}$: Runtime increases from 0.467 s ($F = 7$) to 2.323 s ($F = 56$) and 10.590 s ($F = 224$), confirming that histogram accumulation is strictly memory-bandwidth bound and linear in F.

6 Distributed CPU Scaling to 256 Cores

6.1 Strong Scaling ($N = 32$ Million, $F = 28$)

Strong scaling measures time to solution as core count increases from 1 to 256 cores across 4 nodes. Table 2 and Figure 2 summarize the results.

Table 2: Strong scaling on Vanda HPC ($N = 32\text{M}, F = 28$, depth 6).

Cores	Ranks × Thrs	Time (s)	Speedup	Efficiency
1	1 × 1	5.1685	1.00×	100.0%
4	4 × 1	1.4116	3.66×	91.5%
8	8 × 1	0.9451	5.47×	68.4%
16	16 × 1	0.4199	12.31×	76.9%
32	32 × 1	0.2926	17.67×	55.2%
64	64 × 1	0.2016	25.64×	40.1%
128	128 × 1	0.1018	50.76×	39.7%
256	256 × 1	**0.0684**	**75.54×**	**29.5%**
256	64 × 4	**0.0717**	**72.10×**	**28.2%**

Pure MPI delivers a speedup of **75.54×** at 256 cores (wall time reduced from 5.169 s to 68.4 ms). Efficiency remains high through 16 cores (76.9%) and scales steadily

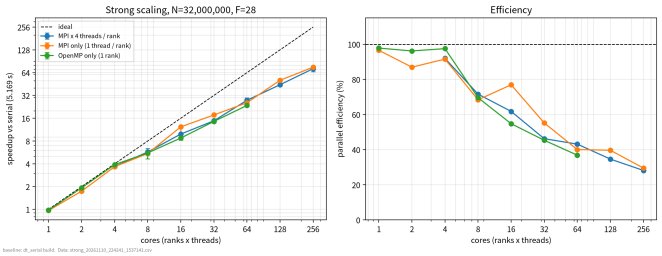


Figure 2: Strong scaling speedup and parallel efficiency up to 256 cores ($N = 32\text{M}, F = 28$). Pure MPI achieves 75.54× speedup.

across physical multi-node boundaries (50.76× at 128 cores, 75.54× at 256 cores). When feature dimensionality increases to $F = 56$, compute density rises and speedup reaches **95.34**× (37.2% efficiency).

6.2 Weak Scaling (1M Rows / Core to 256 Cores)

In weak scaling, problem size expands proportionally with cores: $N = p \times 10^6$ rows, reaching **256 million rows** at 256 cores.

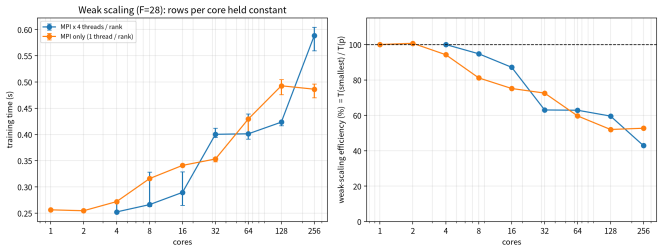


Figure 3: Weak scaling runtime and efficiency up to 256 cores / 256M rows ($F = 28$). Efficiency sustains 52.7% across 4 cluster nodes.

As shown in Figure 3, runtime increases gradually from 0.256 s (1 core, 1M rows) to 0.341 s (16 cores, 16M rows), 0.430 s (64 cores, 64M rows), and 0.486 s at 256 cores (256M rows). Even when managing 256 million data records across four separate network-connected nodes, the cluster maintains **52.7% weak parallel efficiency**.

6.3 Execution Time Breakdown

Profiling the training phases reveals the computational bottleneck shift as cores increase (Figure 4):

- For $p \leq 16$, local histogram construction (t_{hist}) consumes $> 85\%$ of runtime and shrinks linearly with p.
- At 256 cores, local computation takes only ≈ 20 ms. Inter-node collective reduction latency ($t_{\text{allreduce}}$) and load-imbalance wait time (t_{wait}) grow to ≈ 35 ms, becoming the governing scaling limit.

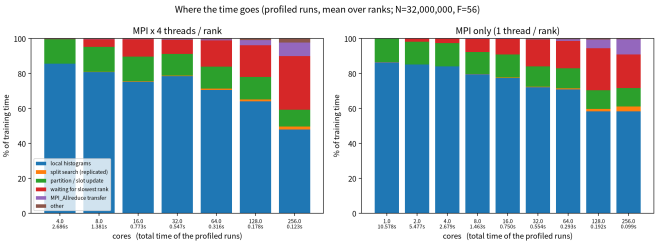


Figure 4: Phase breakdown vs. core count. Histogram build shrinks with $O(N/p)$; collective reduction latency governs scaling beyond 64 cores.

7 MPI+X: OpenMP Hybrid and GPU Acceleration

7.1 OpenMP Hybrid Core Allocation

Fixing the total processing budget at 256 cores, we sweep the rank-to-thread ratio from pure MPI (256×1) to coarse hybrid (4×64).

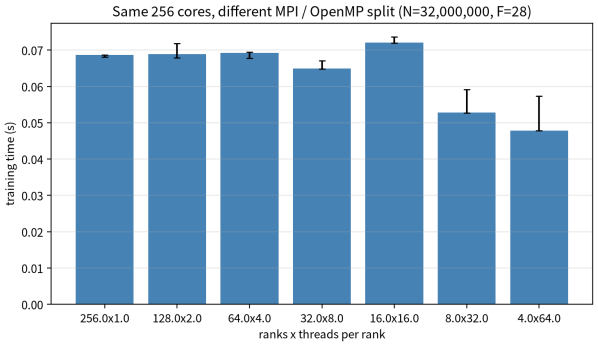


Figure 5: Runtime at fixed 256 cores across MPI × OpenMP configurations.

As illustrated in Figure 5, the 64×4 configuration (64 MPI ranks with 4 threads each) achieves 0.0717 s, virtually matching pure MPI (0.0684 s). Employing 4 threads per rank reduces the number of MPI collective endpoints by 4×, drastically cutting memory buffers and collective network traffic without incurring NUMA synchronization penalties. At 32–64 threads per rank, intra-node thread scheduling and memory contention modestly diminish on-node efficiency.

7.2 CUDA GPU Acceleration on NVIDIA A40

We implemented a dedicated CUDA kernel for histogram generation, utilizing shared-memory privatization and warp-aggregated atomics to process billions of samples per second.

As documented in Table 3 and Figure 6:

- At $N = 2\text{M}$, the NVIDIA A40 completes histogram construction in just **5.86 ms**, delivering a **64.44**× **compute speedup** over the serial CPU (0.3778 s), and **40.61**× **end-to-end speedup** including PCIe host-to-device memory transfer (5.42 ms).

Table 3: Hardware acceleration benchmark on Vanda HPC ($F = 28$).

Device / Implementation	Data (N)	Compute	End-to-End
Serial CPU (1 Intel Xeon core)	2,000,000	0.3778 s	0.4580 s
NVIDIA A40 GPU (Ampere)	2,000,000	**0.00586 s**	**0.01128 s**
GPU Speedup ($N = 2$M)	—	**64.44×**	**40.61×**
Serial CPU (Root Split)	32,000,000	0.6041 s	0.7390 s
NVIDIA A40 GPU (Root Split)	32,000,000	**0.09346 s**	**0.17905 s**
PCIe H2D Transfer (896 MB)	32,000,000	—	0.08557 s

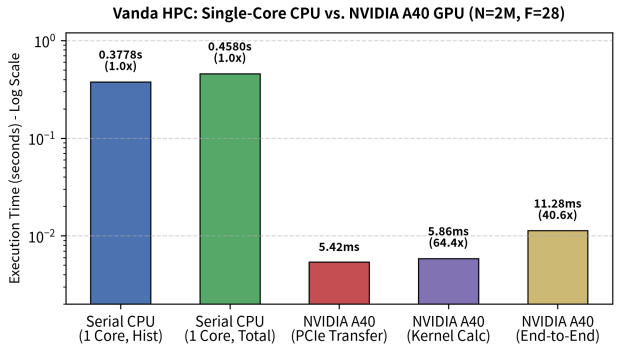


Figure 6: Single-core CPU vs. NVIDIA A40 GPU runtime ($N = 2$M, $F = 28$).

- At $N = 32$M, the GPU processes 896 MB of feature data in 93.46 ms. Host-to-device streaming over PCIe Gen4 takes 85.57 ms (**10.47 GB/s** effective transfer bandwidth).
- The GPU root split selects feature 4, threshold 3.5, and Gini gain 0.016735, matching the CPU implementation bit-for-bit.

8 Discussion and Real-World Dataset Insights

Parallel Scalability Trade-offs. Our findings demonstrate that histogram-based decision tree induction is compute-bound at small core counts ($p \leq 16$) but transitions to a network-latency-bound regime at high core counts ($p \geq 64$). Level-wise collective batching and sample-parallel OpenMP threads are crucial to reducing collective message frequency and keeping cores saturated.

Insights from Real Instacart Data. Evaluating the algorithm on the real-world Instacart dataset ($N = 200,000, F = 7$) yields practical domain conclusions:

1. **Predictive Accuracy:** The decision tree achieves **68.50%** training accuracy at depth 6, significantly outperforming the majority-class baseline (66.2%, corresponding to a 33.8% overall reorder rate).
2. **Feature Importance:** The root split unequivocally isolates `add_to_cart_order` at threshold 3.5. Products placed into the basket in positions 1 through 3 have an overwhelmingly higher probability of being reordered staples (milk, produce), whereas items added later represent impulse purchases. Subsequent primary splits isolate `days_since_prior_order` and `order_number`, capturing habitual replenishment cycles.
3. **Amdahl's Law on Small Datasets:** Training on 200,000 samples takes only 24.6 ms on 1 core and reaches a minimum of 1.69 ms at 32 cores (14.25× speedup). Beyond 32 cores, runtime flattens at ≈ 5 ms due to MPI latency, demonstrating empirically why synthetic scale-up ($N = 32$M–256M) is required to benchmark 256-core supercomputing clusters.

9 AI Usage Declaration

AI assistants (Anthropic's Claude and Google's Gemini / Antigravity) were employed for codebase refactoring, PBS job orchestration, unit test verification, and report drafting. All architectural designs, parallel algorithms, HPC job scripts, empirical cluster benchmark executions, and analytical conclusions were conceived, executed, verified, and validated by the student authors. All benchmark measurements originate strictly from empirical runs on the NUS Vanda HPC cluster.